

Reconhecimento de Números Manuscritos

Projeto de Rede Neural Convolucional com TensorFlow e OpenCV

Visão Geral do Projeto

O projeto consiste na criação de um sistema capaz de reconhecer números escritos a mão e classificá-los em tempo real utilizando uma webcam.

Ele é dividido em duas partes principais:

1. **Treinamento e Validação do Modelo:** O arquivo Jupyter Notebook (number_classifier.ipynb) onde a CNN é projetada, treinada com o dataset MNIST e avaliada.
 2. **Reconhecimento em Tempo Real:** O script (webcam.py) que usa OpenCV para capturar vídeo, processar os quadros (frames) e aplicar o modelo treinado para inferência ao vivo.
-

O Conjunto de Dados: MNIST

O modelo utiliza o **MNIST**, o clássico banco de dados de dígitos manuscritos.

- **Formato:** Imagens de 28x28 pixels em escala de cinza.
 - **Tamanho:** 60.000 amostras de treinamento e 10.000 amostras de teste.
 - **Pré-processamento:** As imagens foram normalizadas dividindo os valores dos pixels por 255.0 (escala [0, 1]) e expandidas para incluir a dimensão de canal (28x28x1), requisito das redes convolucionais.
-

Reprodutibilidade (Uso de Seed)

Para garantir que os resultados do treinamento possam ser replicados fielmente em execuções futuras, o projeto configura uma semente (seed) global (SEED = 42).

Isso afeta componentes estocásticos do sistema, travando a aleatoriedade em:

- Geração de números no Python nativo (`random.seed`).
- Operações matemáticas no NumPy (`np.random.seed`).
- Inicialização de pesos e *Data Augmentation* no TensorFlow (`tf.random.set_seed`).

Desta forma, os experimentos tornam-se determinísticos, auxiliando muito na depuração e avaliação do modelo.

Arquitetura da CNN

O modelo é da classe Sequential e construído em três etapas fundamentais:

1. **Data Augmentation:** Rotação (RandomRotation) e Zoom aleatórios (RandomZoom) para tornar o modelo mais robusto contra pequenas inclinações e tamanhos variados.
 2. **Extração de Características:**
 - Camada Conv2D com 32 filtros + MaxPooling2D.
 - Camada Conv2D com 64 filtros + MaxPooling2D.
 3. **Classificação (Dense Layers):**
 - Flatten (transformação para 1D).
 - Dense com 64 neurônios e ativação ReLU.
 - Dropout (0.5) para mitigação de *overfitting*.
 - Dense de saída com 10 neurônios (0 a 9) e ativação Softmax.
-

Treinamento e Avaliação

Para o treinamento do modelo, as configurações otimizadas foram aplicadas:

- **Otimizador:** Adam.
 - **Loss:** Sparse Categorical Crossentropy.
 - **Early Stopping:** Monitora a métrica `val_loss` para pausar o treinamento caso a rede pare de melhorar após 5 épocas, restaurando os melhores pesos.
 - **Desempenho:** O modelo atinge de forma consistente uma altíssima acurácia (aproximadamente ~99% no conjunto de testes), demonstrando forte capacidade de generalização.
-

Sistema em Tempo Real (webcam.py)

A inferência ao vivo junta o modelo salvo (modelo.h5) com a poderosa biblioteca de Visão Computacional **OpenCV**.

- Uma janela interativa abre com a câmera capturando vídeo.
 - Um retângulo verde é desenhado para delimitar a **Região de Interesse (ROI)** onde o usuário deve posicionar o número escrito.
 - Apenas o conteúdo dentro do quadro é isolado para análise da inteligência artificial.
-

Pré-processamento na Câmera

Para que o modelo entenda a imagem da câmera, precisamos deixá-la exatamente igual ao padrão do MNIST (números brancos com fundo preto em 28x28).

O fluxo na ROI capturada é o seguinte:

1. Conversão para tons de cinza (cv2.COLOR_BGR2GRAY).
2. Suavização para remoção de ruídos (cv2.GaussianBlur).
3. **Binarização Invertida** (cv2.THRESH_BINARY_INV): Torna o fundo preto e os traços brancos através do limiar.
4. Redimensionamento para 28x28 pixels.
5. Normalização (divisão por 255.0) e remodelagem para (1, 28, 28, 1).

Inferência e Interface

Com os dados formatados, a previsão é feita:

- O array passa pelo modelo.predict.
- Calculamos a classe provável (np.argmax) e a confiança (np.max).
- Se a **confiança for maior que 50%**, o programa insere o texto indicando o número e sua porcentagem logo acima do retângulo verde.

Feedback Duplo: O sistema exibe duas telas: a visualização convencional da webcam e uma segunda janela ampliada chamada "Visão do Modelo", mostrando os exatos pixels 28x28 binarizados que a CNN enxerga.

Conclusões

O projeto comprova ser um excelente estudo de caso de ponta a ponta:

- Da exploração dos dados, elaboração e treinamento da arquitetura Deep Learning.
- Ao deployment funcional em tempo real combinando manipulação avançada de imagens no OpenCV.

A abordagem de Thresholding e simulação visual do MNIST garante que o modelo reconheça traços do mundo real de forma robusta e altamente confiável.